Sistemas Distribuidos

PBFT

Rodrigo Santamaría

PBFT

- Objetivo
- Requisitos básicos
- Requisitos adicionales
- Despliegue
- Entrega y defensa

PBFT

Objetivo

- Implementar un método de consenso distribuido basado en el algoritmo *PBFT* (tema 5, diap. 84)
  - En el libro, capítulo 4.4.3 (pág. 51)
- El sistema:
  - A través de un cliente, propondrá periódicamente valores a todos los oyentes, para llegar a un consenso
  - Los procesos, que actuarán de oyentes, tendrán un porcentaje de probabilidad de error (fallos bizantinos)

PBFT

- Objetivo
- Requisitos básicos
- Requisitos adicionales
- Despliegue
- Entrega y defensa

Consenso distribuido

Requisitos básicos

- C1 (**Terminación**): cada proceso correcto ha de fijar su variable de decisión
- C2 (**Acuerdo**): el valor de decisión para todos los procesos correctos es el mismo
- C3 (**Integridad**): si todos los procesos correctos han propuesto el mismo valor, entonces cualquier proceso correcto en el estado decidido ha elegido dicho valor
- No debe ocurrir
  - Espera ocupada
  - Interbloqueos
  - Inanición

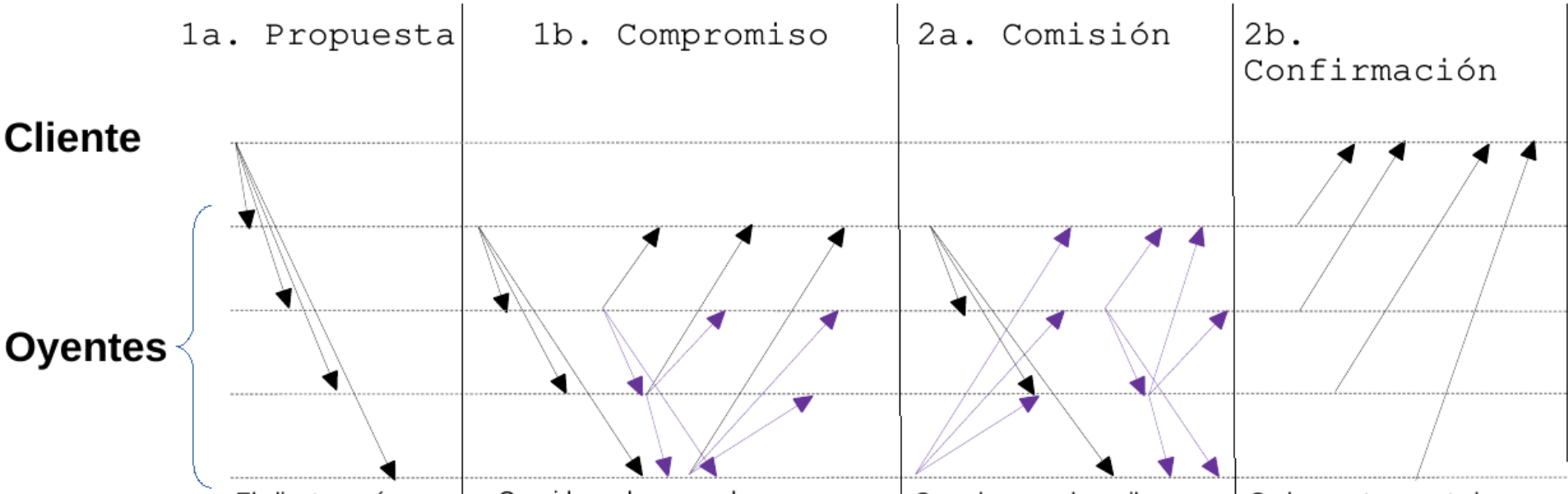
PBFT

Requisitos: entorno de programación

- El lenguaje de programación y bibliotecas asociadas **no es un requisito**
  - Libertad de elección y diseño, mientras se cumplan el resto de requisitos
  - Se dan las pautas y ayuda en Java+Jersey, pero no es obligatorio usar este esquema
- El **contexto sí es un requisito**: la práctica se defenderá en los ordenadores del Laboratorio de Informática/Aula SUN en el sistema operativo Linux instalado.

PBFT

Esquema general



El cliente envía una actualización a todos los nodos (no hay un postulante en el que se confíe para iniciar el proceso).

Considerando que no hay propuestas concurrentes o caídas en los nodos, los oyentes se comprometen con la actualización *con todos los demás nodos de la red, no sólo el postulante*. Esta todavía no se lleva a cabo.

Cuando un nodo recibe una mayoría de compromisos (en este caso 3), comete la actualización y lo comunica a los demás

Cada oyente comete la actualización y responde al cliente. Este considerará la actualización realizada si recibe confirmación de una mayoría (quórum) de nodos

PBFT

- Objetivo
- Requisitos básicos
- Requisitos adicionales
- Despliegue
- Entrega y defensa

PBFT

Repetición de código = suspenso

- El uso de **repetición de código** es motivo de **suspenso**
- Es una práctica de programación:
  - Deficiente
  - No escalable
  - Fuente de errores
  - Impropia de un ingeniero

```
Proceso p1=new Proceso(1);
Proceso p2=new Proceso(2);
Proceso p3=new Proceso(3);
Proceso p4=new Proceso(4);

p1.start();
p2.start();
p3.start();
p4.start();
```

```
const int maxP=4;
List<Proceso> lista=
       new List<Proceso>();

for(int i=0;i<maxP;i++)
  {
  Proceso p=new Proceso(i);
  p.start();
  lista.add(p);
  }
```

PBFT

Requisitos adicionales (opcionales)

- Interfaz
  - Minimizar métodos y parámetros
  - Claridad, documentación
- Arquitectura
  - Minimizar nº de clases
  - Despliegue sencillo
- Características
  - **Extensibilidad**: facilidad de cambiar a otros nodos/máquinas
  - **Escalabilidad**: facilidad de ampliar a más nodos/máquinas
  - **Rendimiento**: tiempo en llegar a un acuerdo, afinar timeouts
  - **Tolerancia**: fallo de muchos/todos los procesos, recuperación en medio de elección, etc.

PBFT

- Objetivo
- Requisitos básicos
- Requisitos adicionales
- Despliegue
- Entrega y defensa

PBFT

Clases

- Vamos a necesitar 3 clases
  - **Proceso**: implementada por cada uno de los hilos que participarán en el consenso distribuido
  - **Cliente**: ejecutada en una de las tres máquinas, ofrecerá una interfaz para permitir modificar el sistema.
  - **Servicio**: mantiene la lista de procesos de cada máquina e implementa servicios REST para acceder a sus distintos métodos (sólo necesaria si trabajamos con un único servidor/puerto por máquina)

Consenso distribuido

Proceso

- Clase que extiende de `Thread`
- Atributos
  - `id`: identificador del proceso
  - `variable`: entero positivo sobre la que se aplica el consenso
  - `error`: booleano, si true el proceso funciona con fallos bizantinos (por defecto a false)
  - `compromisos`: array de los valores sobre los que se ha recibido compromiso
  - `comisiones`: array de los valores sobre los que se ha recibido comisión
- Métodos
  - `propuesta(v)` modifica el valor de la variable a un valor que signifique "no decidido" (null, 0 o -1, p. ej.), y vacía el array de compromisos y de comisiones. Multidifunde `compromiso(v)` o `compromiso(random)` si `error=true`
  - `compromiso(v)` modifica el estado de los compromisos recibidos. Si hay quórum de compromisos para un valor, emite `comision(v)`
  - `comision(v)` modifica el estado de las comisiones recibidas. Si hay quórum de comisiones, emite `confirmacion()` al cliente.
  - `confirmación()` de que el valor se ha cambiado. Si hay quórum de confirmaciones, el cliente notifica al usuario que el cambio se ha realizado.*

* Hay varias formas de implementar la confirmación en el cliente:
1) La respuestas a la petición síncrona al servicio propuesta
2) El evento que se dispare con la respuesta a la petición asíncrona al servicio propuesta
3) También se puede hacer un servicio en el Cliente que implemente el método `confirmación`

Consenso distribuido

Proceso: fallos bizantinos

- Simularemos que la fase 1b puede sufrir errores bizantinos
  - Si `error==true`, el nodo multidifunde como compromiso valores aleatorios entre 0 y 100.
    - Se puede ‘excluir’ el valor propuesto por el cliente de entre los valores aleatorios, aunque tampoco pasa nada por dejarlo, si se tiene todo bien documentado
    - El nodo, aunque haga la multidifusión mal, se compromete bien con el valor propuesto por el cliente.
      - Aunque si se prefiere, se puede hacer al revés: comprometer mal y multidifundir bien.

PBFT

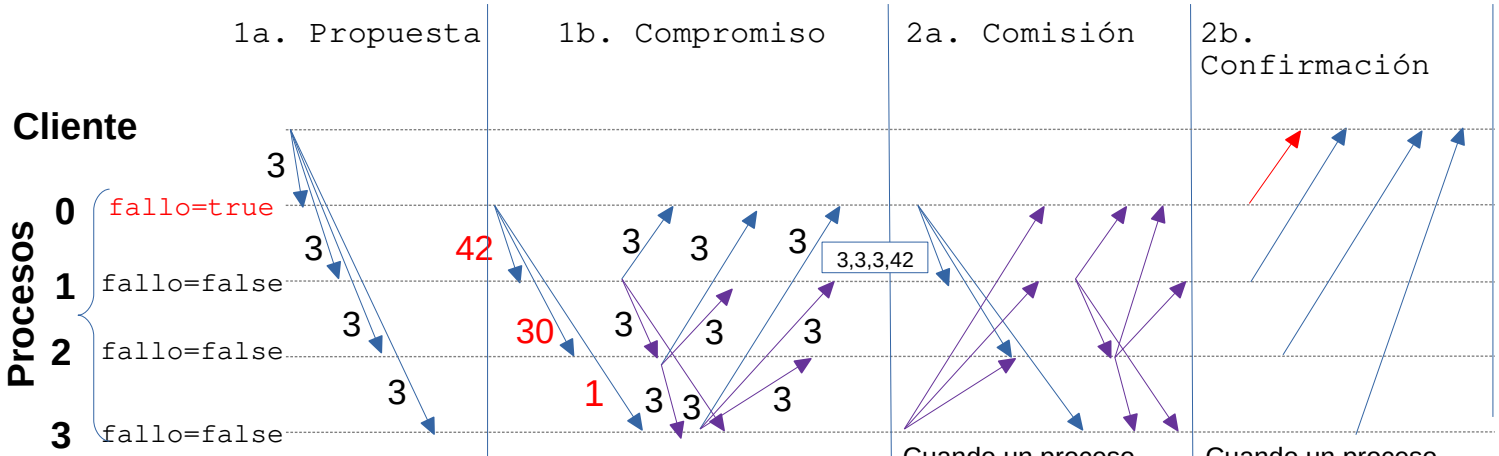
Cliente

- Clase que permite la interacción con el usuario a través de una interfaz sencilla con las siguientes opciones (entre paréntesis la combinación de teclas para cada opción):
  - `Fallo (fN)`: modificar el estado de fallo del proceso N
    - Si estaba a true lo pone a false, o viceversa
  - `Cambiar valor(sX)`: proponer un cambio del valor de la variable a X
  - `Estado (s)`: muestra una tabla separada por tabuladores indicando, para cada nodo, su id, el valor de su variable, los compromisos recibidos, y su estado de fallo (ver ejemplo abajo)
  - `Ayuda (h)`: muestra las opciones disponibles y sus combinaciones de teclas.

id	var	compromisos	error
1	6	6,6,6,32,1	true
2	6	3,5,6,6,6	false
3	6	6,6,6,77,2	false
4	-	2,7,6	false
5	-	6,6	true

PBFT

Esquema general: fallos bizantinos



El cliente suponemos que nunca tiene errores. En este ejemplo, uno de los procesos (P0) envía propuestas aleatorias a cada proceso.

Cada proceso retransmite al resto de procesos el valor con el que se compromete, que será aquel recibido del cliente.

Cada proceso va modificando una estructura interna de compromisos recibidos para cada valor. Por ejemplo, P1 terminará con 3 compromisos para el valor 3 (del propio P1 recibido del cliente, de P2 y P3) y uno para el valor 42 recibido del proceso P0 que está fallando.

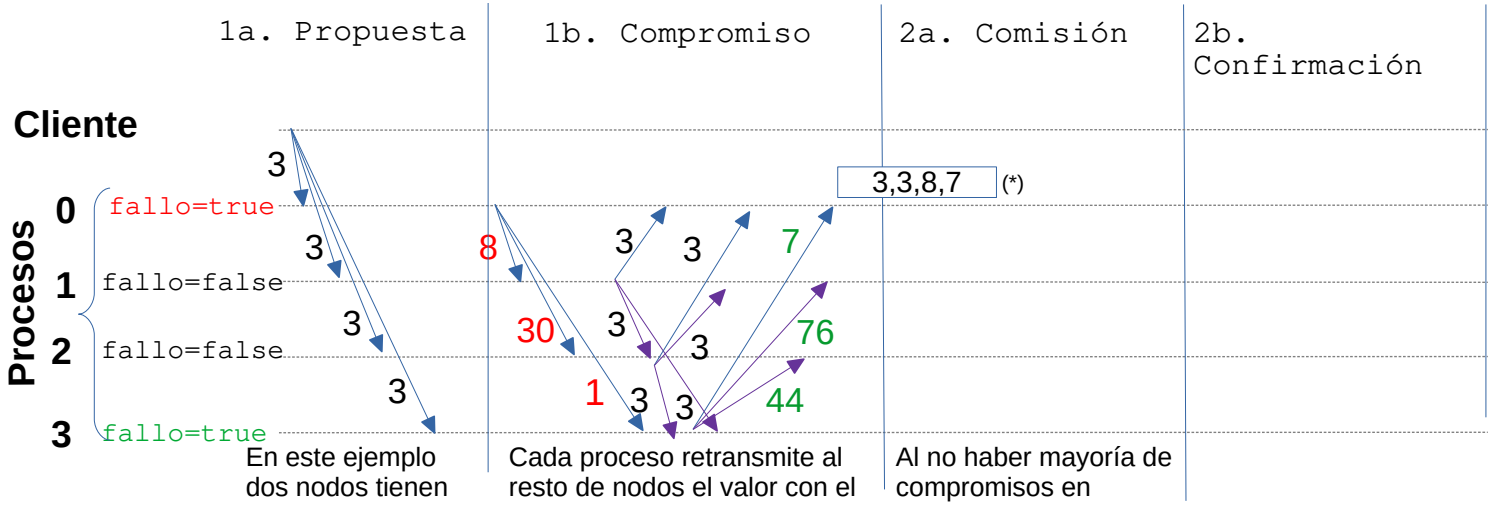
Cuando un proceso recibe una mayoría de compromisos para un valor (en este caso 3 de 4), comete la actualización, es decir cambia el valor de la variable, y lo comunica a los demás.
Los procesos, al recibir 3 compromisos para el valor 3, aunque haya otro valor discrepante, cometen la operación para el valor mayoritario (3).

Cuando un proceso recibe comisiones de una mayoría de procesos, manda un mensaje de confirmación al cliente. Este considerará la actualización realizada si recibe confirmación de una mayoría (quórum) de nodos

Se podría implementar que el nodo bizantino no emita confirmación o comisión, opcionalmente, aunque no afectaría al funcionamiento general.

PBFT

Esquema general: fallos bizantinos



En este ejemplo dos nodos tienen fallos arbitrarios. P0 y P3

Cada proceso retransmite al resto de nodos el valor con el que se compromete, que será aquel recibido del cliente.

En este caso P3 también falla, así que dice que se compromete con valores aleatorios

Cada proceso va modificando una estructura interna de compromisos recibidos (*). En este caso, ningún valor obtiene quórum (hay 2 procesos con fallos, el 50%)

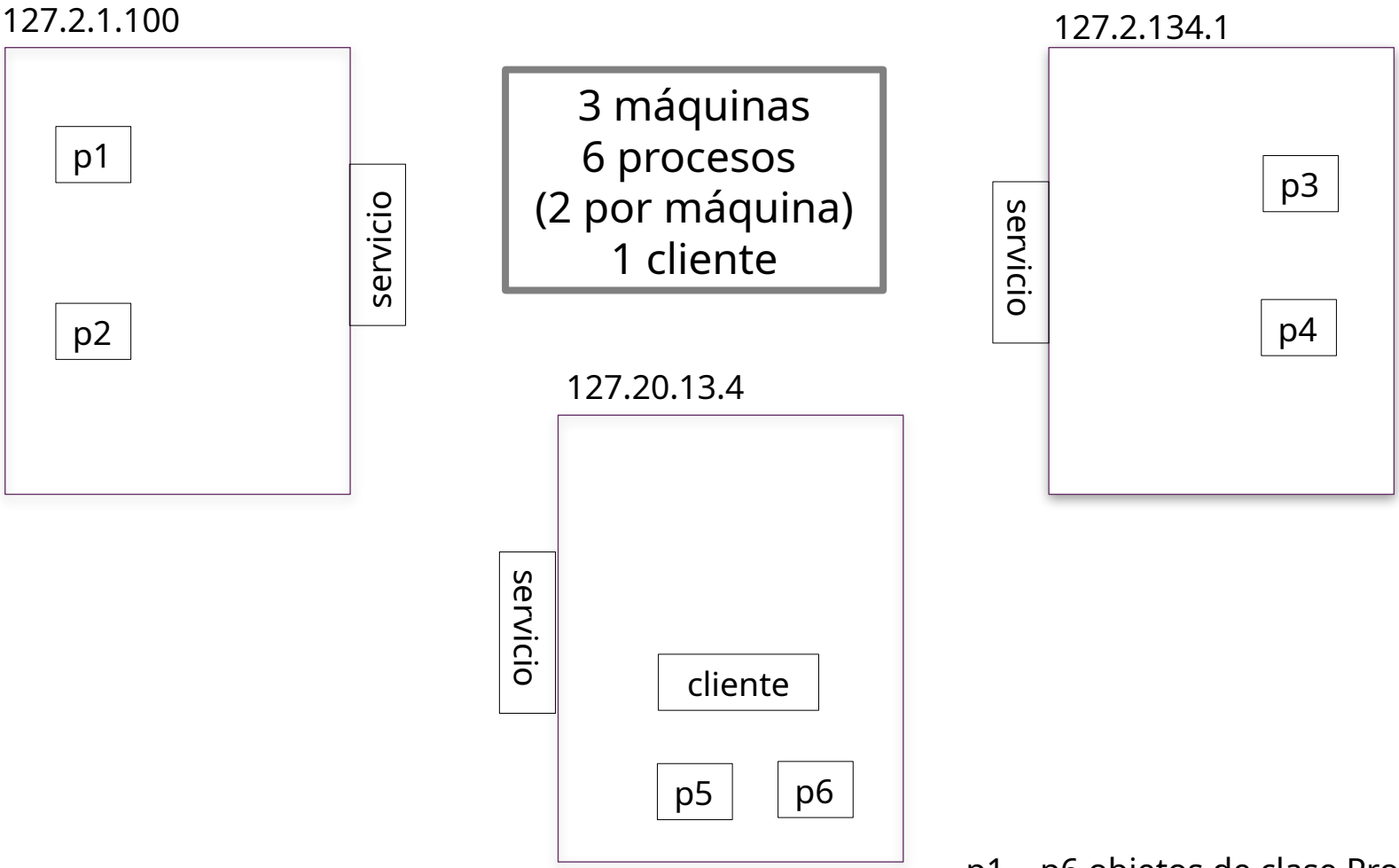
Al no haber mayoría de compromisos en ningún nodo, ninguno de los nodos pasa a fase 2. La variable no se modifica.

Se podría implementar una política de reenvíos por parte del cliente en caso de que no reciba confirmaciones tras un tiempo, pero no vamos a hacerlo.

Se puede implementar un mensaje de confirmación negativa, o bien que el cliente espere por un tiempo por la respuesta y en caso de no recibirlas, considere que no ha habido consenso.

PBFT

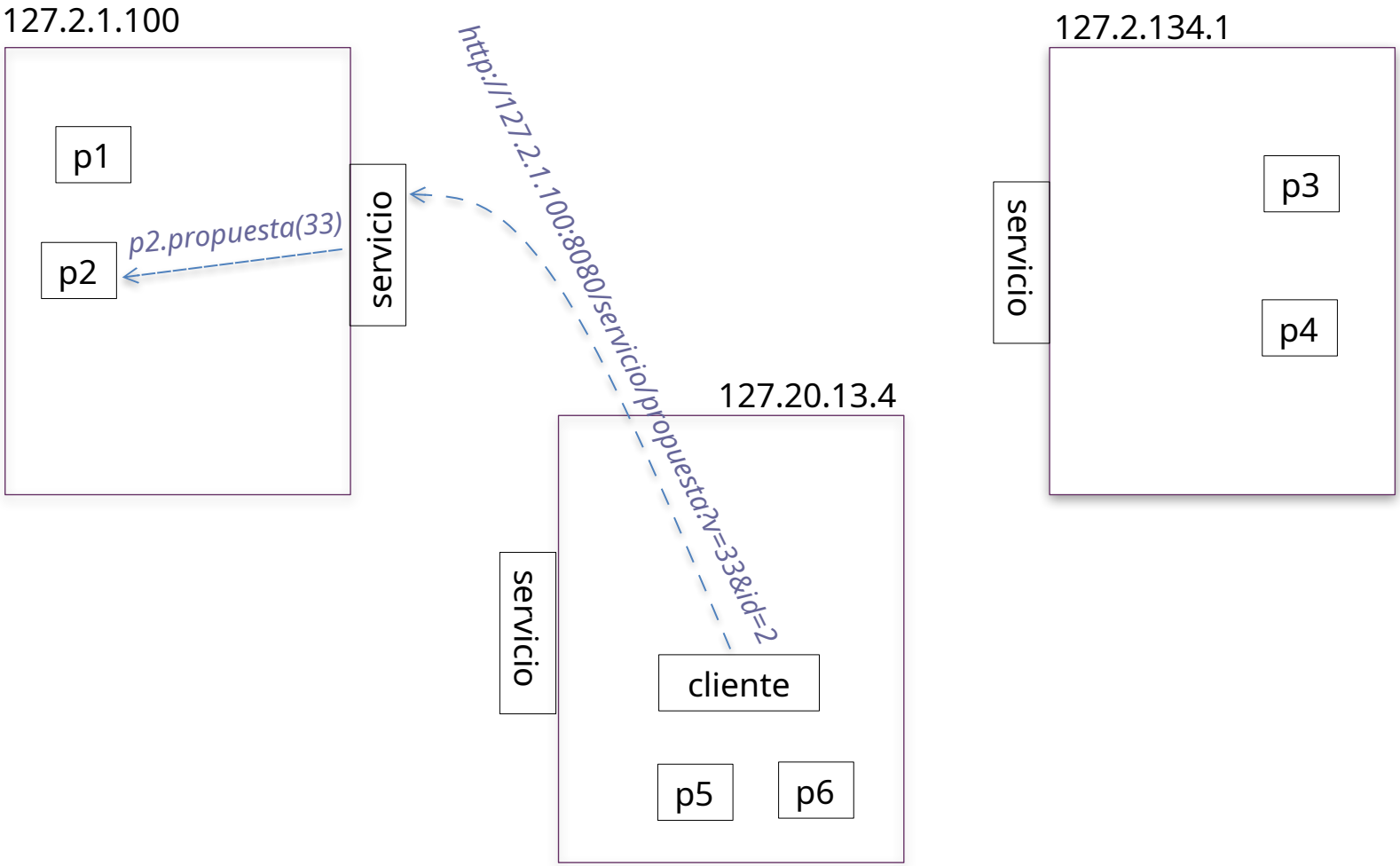
Servicio: Despliegue



p1… p6 objetos de clase Proceso

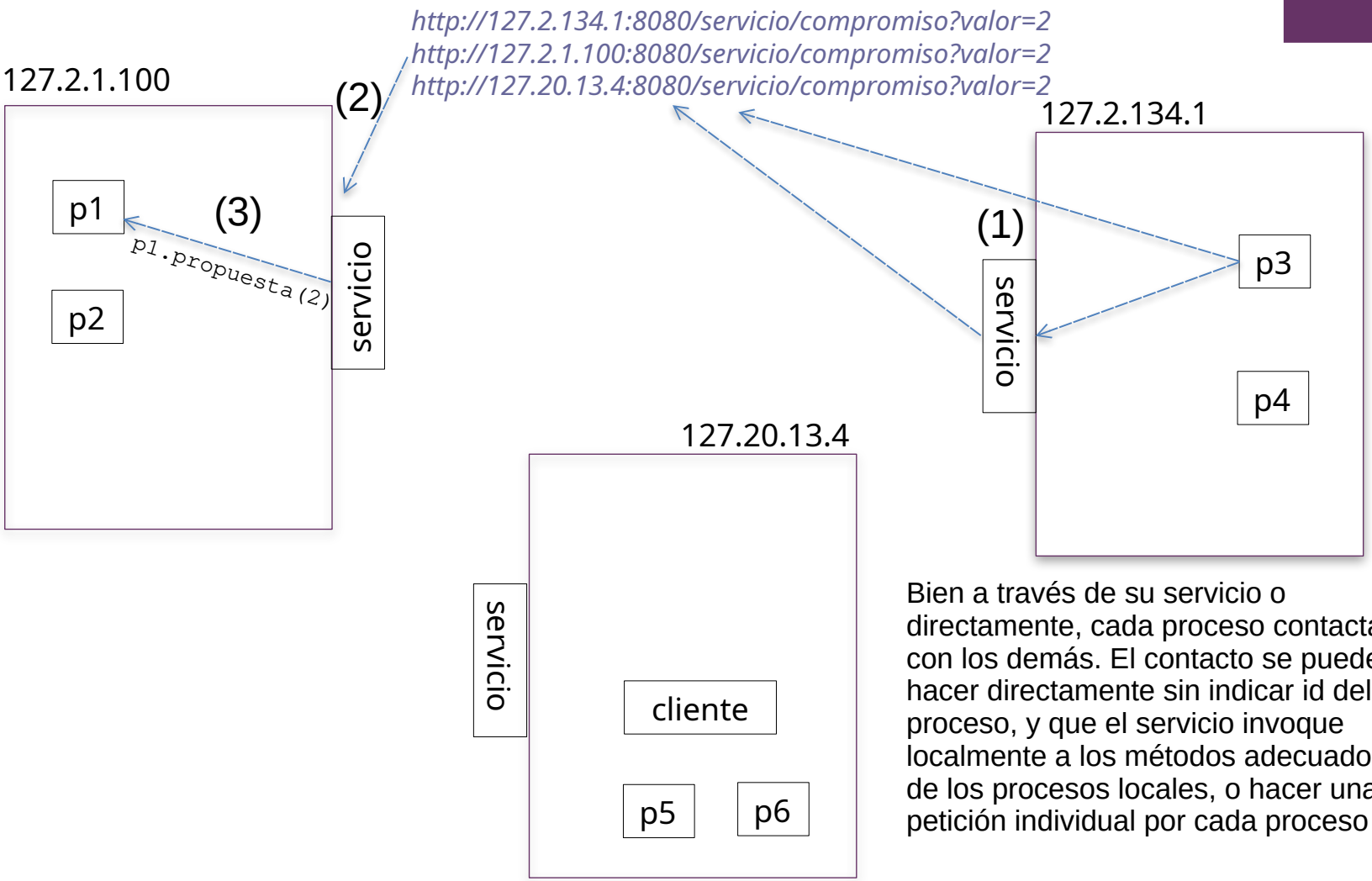
PBFT

Servicio: Despliegue



PBFT

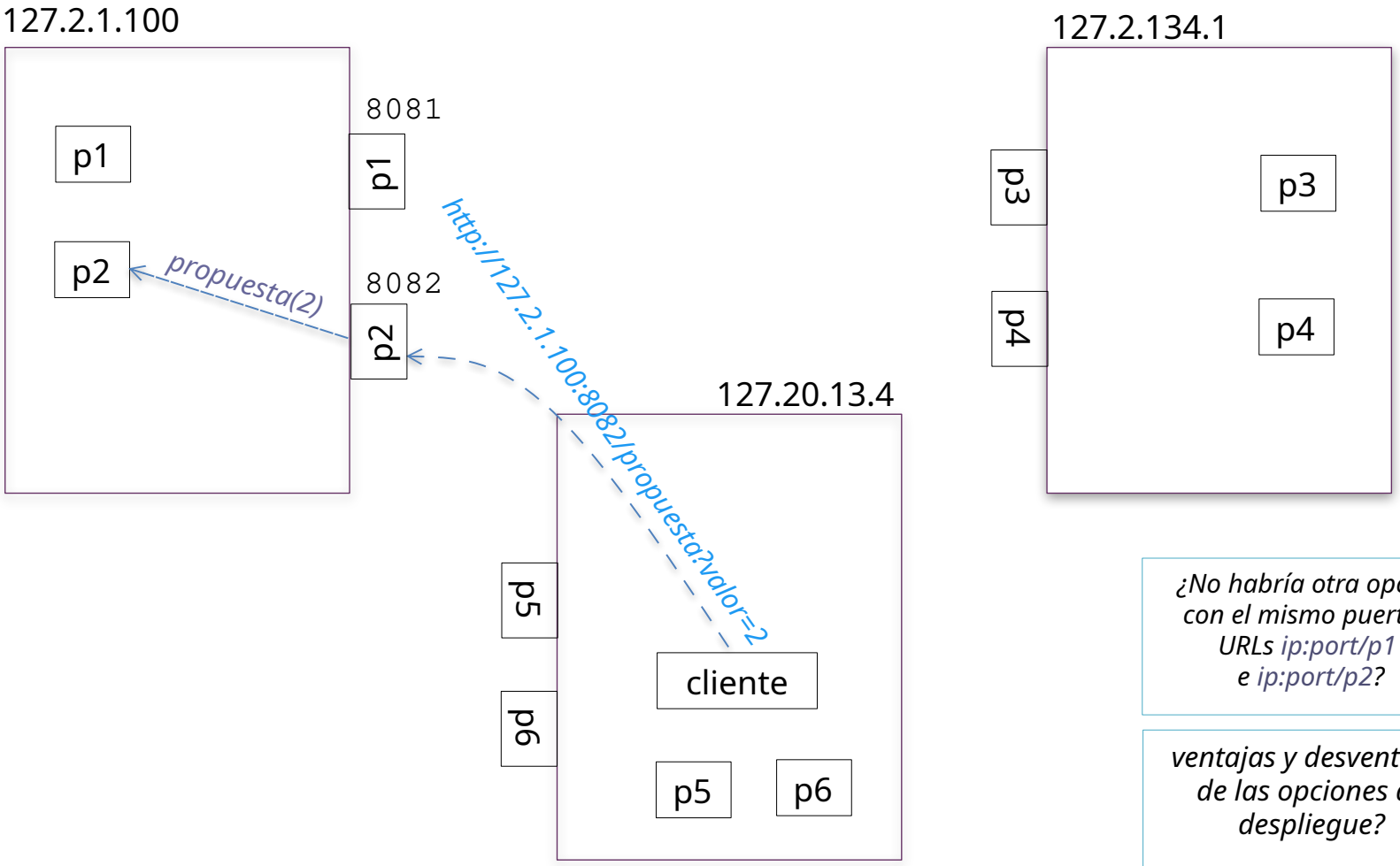
Servicio: Despliegue



Bien a través de su servicio o directamente, cada proceso contactará con los demás. El contacto se puede hacer directamente sin indicar id del proceso, y que el servicio invoque localmente a los métodos adecuados de los procesos locales, o hacer una petición individual por cada proceso

PBFT

Despliegue: opción sin servicio



¿No habría otra opción con el mismo puerto y URLs ip:port/p1 e ip:port/p2?

ventajas y desventajas de las opciones de despliegue?

PBFT

- Objetivo
- Requisitos básicos
- Requisitos adicionales
- Despliegue
- Entrega y defensa

PBFT

Entrega

- La entrega se realizará, por parejas*, a través de Studium
- Ambos miembros de la pareja entregarán un fichero **tar.gz**
  - Con nombre *Nombre1Apellido1Nombre2Apellido2.tar.gz*
  - Contendrá
    - Las clases `Proceso`, `Cliente` y `Servicio`**
    - Cualquier otra clase, script, etc. necesario para la ejecución
    - No es necesario incluir bibliotecas externas (jersey, etc.)
- Fecha límite de entrega: **8 de mayo a las 23h**

* Salvo circunstancias excepcionales
**Esta última clase puede obviarse dependiendo del tipo de despliegue elegido

PBFT

Defensa

- El horario de defensa se publicará tras la entrega
- Las defensas tendrán lugar la semana siguiente a la entrega
- Se realizará en el Laboratorio de Informática / Aula SUN
  - Cada defensa durará unos 20 minutos, en los que:
    - Se probará exhaustivamente la práctica
    - Se examinará el código fuente
    - Se harán preguntas a ambos miembros de la pareja